NAME: Moshitha kolanjiyappan
REG NO:192421205

Industry Problem-Solving Task: Motion Recognition Assessment

1. Industry Challenge Scenario

You are an engineer at an automotive factory. Your team is replacing physical control buttons with an OpenCV-based Motion and Gesture Recognition System so workers can control machinery hands-free. The system captures video streams, computes frame-to-frame differences or optical flow to isolate motion, and uses a lightweight neural network to classify the motion into three operational commands: START (Wave Hand), STOP (Raise Fist), and NEXT_STEP (Swipe Left).

The factory floor presents severe real-world challenges: fluctuating overhead lighting, dust particles creating pixel noise, and a requirement that the system must run on a low-power edge processor (Raspberry Pi 5) directly at the workstation. To solve this, your team tests four different OpenCV motion pre-processing pipelines across a validation set of 2,000 video clips (sampled evenly across all three gestures).

Table 1: OpenCV Motion Processing Pipeline Configurations

Configuration ID	OpenCV Pre-processing Approach	Target Resolution	Structuring Element Size (cv2.morphologyEx)	Frame Skipping Factor
Pipeline Alpha	Absolute Frame Differencing (cv2.absdiff) + cv2.threshold	640 x 480	None	None (Process every frame)
Pipeline Beta	Background Subtraction (cv2.createBackgroundSubtractorMOG2)	640 x 480	cv2.MORPH_RECT (5 x 5)	Skip 1 frame in 2
Pipeline Gamma	Dense Optical Flow (cv2.calcOpticalFlowFarneback)	320 x 240	None	Skip 2 frames in 3

Pipeline Delta	Background Subtraction (cv2.createBackgroundSubtractorMOG2)	320 x 240	cv2.MORPH_RECT (3 x 3)	None (Process every frame)

Table 2: Industry Deployment Validation Performance

Pipeline ID	Latency per Frame (ms)	False Trigger Rate	Correctly Classified Gestures	Completely Missed Gestures	Average CPU Temp
Alpha	4.2 ms	34.2%	1,410	180	48 deg C
Beta	22.1 ms	2.1%	1,820	60	62 deg C
Gamma	41.5 ms	0.8%	1,910	10	81 deg C (Throttling)
Delta	9.8 ms	4.5%	1,890	25	54 deg C

2. Questions for Students

Question 1: Technical Analysis of Algorithm Failure
Pipeline Alpha achieves incredibly low latency (4.2 ms) but suffers from a disastrous False Trigger Rate of 34.2%. Explain why a basic cv2.absdiff approach fails on a real-world factory floor. What mathematical/statistical changes does MOG2 (used in Beta and Delta) introduce to solve this specific flaw?

Question 2: Hardware Constraints & Trade-offs
Pipeline Gamma yields the absolute highest accuracy (1,910 correct gestures) and a sub-1% false trigger rate. However, the plant manager refuses to deploy it because the CPU temperature reaches 81 deg C, causing thermal throttling. Break down the algorithmic complexity of Dense Optical Flow vs. Background Subtraction to explain why Gamma causes such high computational load despite operating at a much
lower image resolution (320 x 240).

Question 3: Executive Engineering Recommendation
The factory's safety standard states that a STOP gesture must never be missed, and the system must
maintain a processing speed of at least 30 Frames Per Second (FPS) to ensure real-time machinery

control.
1. Calculate the maximum allowable latency per frame in milliseconds to meet the 30 FPS requirement.
2. Based on your calculation and the data tables, which pipeline configuration (Alpha, Beta, Gamma, or
Delta) should be deployed on the factory floor? Justify your choice by explicitly citing metrics from both
tables.

Question 1: Technical Analysis of Algorithm Failure

Answer

Pipeline Alpha uses cv2.absdiff() to calculate the absolute pixel difference between two consecutive frames. The mathematical operation is:

$$\mathbf{D(x,y) = |I_t(x,y) - I_{t-1}(x,y)|}$$

where I_t represents the current frame and I_{t-1} represents the previous frame.

The main limitation of cv2.absdiff() is that it treats almost every pixel intensity change as motion. On a real-world automotive factory floor, pixel values can change because of fluctuating overhead lighting, dust particles, shadows, reflections, and camera sensor noise. These changes may occur even when a worker is not performing a gesture.

For example, when factory lighting flickers, many pixels may become brighter or darker at the same time. cv2.absdiff()detects these changes as significant frame differences. Similarly, moving dust particles can produce small regions of changing pixels. When these differences are passed through cv2.threshold(), they may be interpreted as foreground movement and generate false gesture detections.

This explains why Pipeline Alpha has a very high **False Trigger Rate of 34.2%**, despite having a very low latency of only **4.2 ms per frame**.

MOG2 Background Subtraction

MOG2, implemented using:

cv2.createBackgroundSubtractorMOG2()

uses a statistical background model instead of simply comparing two consecutive frames.

MOG2 models each pixel using a **Mixture of Gaussians (MoG)**. A simplified mathematical representation is:

$$\mathbf{P(x) = \Sigma\, w_i\, N(x;\, \mu_i,\, \sigma_i^2)}$$

where:

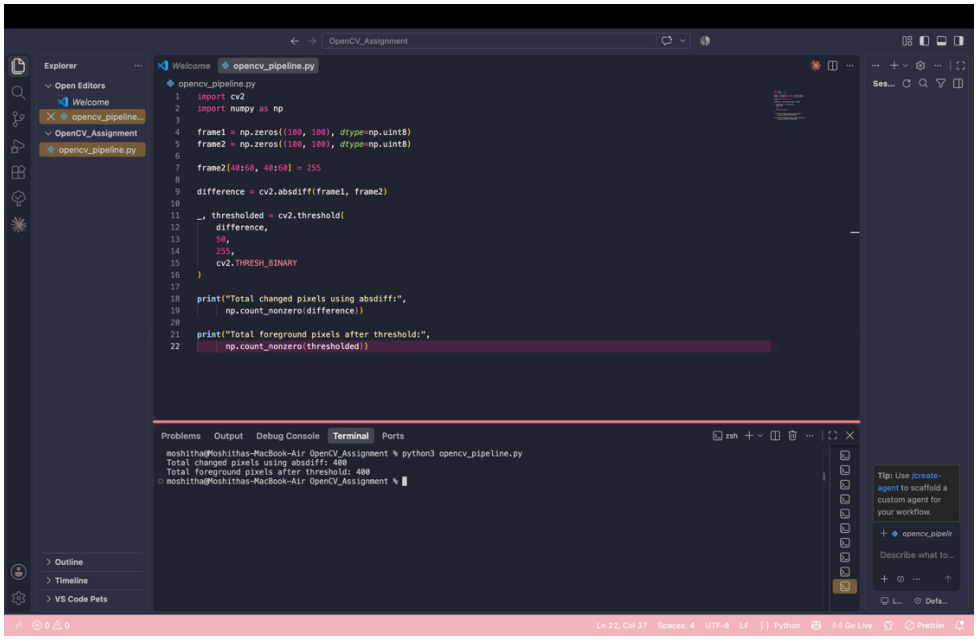
- w_i = weight of the Gaussian component
- μ_i = mean pixel intensity
- σ_i^2 = variance
- N = Gaussian probability distribution

The model maintains a history of pixel values and learns what normally belongs to the background. Therefore, gradual lighting changes and small environmental variations are less likely to be classified as foreground motion.

As a result, MOG2 is more suitable for a factory environment than simple frame differencing.

cv2.absdiff() simply subtracts raw pixel intensity values between adjacent frames. Environmental variations such as **light flicker, passing dust, shadows, reflections, and sensor noise** can cause massive frame-wide difference spikes, producing false positives.

MOG2 models each background pixel independently using a **Mixture of Gaussians**. It retains a statistical run-history distribution, making it highly resilient to continuous small changes, variable noise, and lighting shifts.




MOTION DETECTED
FPS: 462.54
Pipeline Delta - MOG2